

# Deep Learning for MEG Brain-State Classification

**Cem Inal**

Graduate School of Natural Sciences  
Utrecht University  
4627601  
c.inal@students.uu.nl

**Ydo Bergstra**

Graduate School of Natural Sciences  
Utrecht University  
1315609  
y.t.bergstra@students.uu.nl

**Abbas Abbasli**

Graduate School of Natural Sciences  
Utrecht University  
2270293  
a.q.abbasli@students.uu.nl

**Thijmen van der Meijden**

Graduate School of Natural Sciences  
Utrecht University  
1670786  
t.vandermeijden@students.uu.nl

**Abstract**—This report investigates deep learning for four-class brain-state classification from magnetoencephalography (MEG) recordings. The task is evaluated in both intra-subject and cross-subject settings, where the latter requires generalization to unseen individuals. First, we implement a faithful implementation of AA-CascadeNet based on Abdellaoui et al. (2020) [4], however with 2.1M parameters this overfits (training accuracy reaches 1.0, validation stays at chance) on our data. This motivates compact redesign(s) that match model capacity to data capacity to be able to achieve generalisation. Second, we find that contrary to common MEG practice [3] a [1, 45] Hz band-pass *improves* within-subject accuracy but *collapses* cross-subject accuracy to chance. This indicates that the subject-transferable task information lives in the broader frequencies. We therefore retained a more minimal pipeline. Third, our results point that on the harder, cross-subject, task the compact models perform similar to a strong published Transformer baseline (EEG Conformer [5]), and a Transformer back-end (CascadeFormer) marginally edges out the recurrent back-end. Dominant factor in this experiment is the match between model capacity and the small, and especially subject-sparse, data, not necessarily the back-end.

**Index Terms**—MEG, brain-state classification, CNN, BiLSTM, Attention, Transformer, cross-subject generalization

## I. INTRODUCTION

Magnetoencephalography (MEG) is a non-invasive neuroimaging technique that measures the magnetic fields produced by neural activity in the brain. Because MEG recordings capture brain activity with high temporal resolution, they are useful for studying how different cognitive and physical states are reflected in neural signals. In this project, the goal is to use deep learning to classify MEG recordings into four brain states: resting state, math/story task, working memory task, and motor task. Each recording consists of measurements from 248 magnetometer sensors over 35,624 time steps, making the data high-dimensional and temporally complex.

The main challenge in this task is to learn meaningful patterns from MEG time-series data while dealing with noise, large input sizes, possible differences between subjects while having access to little data. This is especially important because brain signals can vary substantially from one individual to another. Therefore, this project considers two classification

settings: intra-subject classification and cross-subject classification. In the intra-subject setting, the model is trained and tested on data from the same subject, while in the cross-subject setting, the model is trained on two subjects and evaluated on three unseen subjects. Comparing these settings allows us to examine not only how well the model learns task-specific MEG patterns, but also how well these patterns generalize across individuals. This report therefore has two main threads. The first is methodological: we correct preprocessing and evaluation choices that initially caused numerical instability and data leakage, and we show that a standard band-pass filter harms cross-subject transferability. The second is architectural and/or capacity-oriented: we demonstrate a faithful AA-CascadeNet, find that it is too large and design compact CNN-recurrent and Transformer-based alternatives to test whether they are able to generalise.

## II. RELATED WORK

**Compact CNNs for EEG/MEG.** EEGNet [1] forms the baseline. This is a two-block depthwise-separable CNN. A temporal convolution for frequency-band filtering that is followed by a depthwise spatial convolution over different (sensor) channels. It is efficient, at the cost of spatial expressiveness as depthwise separable convolutions decouple spatial and temporal weighing. This is further demonstrated by ShallowConvNet and DeepConvNet [2], where fully parametrized spatial convolutions improve classification on EEG. They also introduced sliding-window training with a prediction averaging, which we mirror. LF-CNN and VAR-CNN [3] applied these ideas to MEG, where they showed that spatial filtering improves accuracy and interpretability. The broader landscape of deep learning for EEG/MEG is surveyed by Roy et al. [6].

**Reference model.** AA-CascadeNet [4] extends the CNN front-end with two unidirectional LSTMs and global attention. One-second windows are mapped onto  $20 \times 21$  sensor mesh, split into parallel streams that pass through *independent* convolutional stems, followed by the LSTM layers (and global attention). The model explicitly models long-range temporal

dependencies (after spatial filtering), which is why we chose to work with this model.

**Transformer-based decoding.** EEG Conformer [5] replaces the recurrent back-end with Transformer encoder layer, but retains a compact convolutional embedding. Song et al. (2023) report improvements over RNNs on EEG datasets. We use it as a Transformer baseline, and it performs best on within-subject (intra) setting on MEG, but does not dominate the cross-subject setting.

### III. APPROACH

#### A. Data Preprocessing

Each MEG recording consists of 248 sensor channels and 35,624 temporal samples, stored as an array. Due to the extremely small magnitude of the recorded magnetic signals, a preprocessing pipeline was applied before model training.

Our first preprocessing included subtracting the mean, divide by standard deviation and stabilize with  $\varepsilon = 10^{-8}$ . This failed entirely on our data, as the raw signal values are  $\sim 10^{-12}$  T, so our epsilon dominated the denominator, canceling out normalization. Default hyperparameter values implicitly assume data on the order of unity. We found that they must be re-examined whenever the data scale differs by (many) orders of magnitude. We handled it by dropping epsilon entirely, and handle dead channels explicitly, whilst computing mean and standard deviation in float64 to avoid floating-point *underflow*. Leading to the following:

##### *Corrected pipeline*

1. Downsample by a factor of 8, using `scipy.signal.decimate`. This applies an anti-aliasing low-pass filter before subsampling, which dropped sampling rate significantly (2034 Hz to approximately 254 Hz). All brain-rhythm bands are still within Nyquist limit.

2. Channel-wise Z-scoring. Mean and standard deviation are computed in float64 for each channel independently. Dead channels (zero variance) are zeroed rather than divided. The normalized output is cast to float32, for efficiency. Files are normalized by their own statistics only.

**Band-pass** MEG practice often adds a [1, 45] Hz band-pass to suppress drift and line noise [3]. We evaluated this (see Results, Table I, and found a trade-off: band-pass improves intra-subject accuracy but collapses cross-subject accuracy (to chance). We therefore keep the more minimal, broadband pipeline.

#### B. Window-Based Dataset Construction

Rather than processing an entire MEG recording at once, each recording was divided into fixed-length temporal segments of one second, resulting in samples of shape  $248 \times T$ , where  $T = 254$  represents the number of temporal samples after downsampling. After downsampling, each recording contains approximately 4453 samples, yielding 17 non-overlapping windows per file.

Each extracted window inherited the class label of its corresponding recording, as each recording has a single sustained task condition throughout. Four classes were considered:

resting state, motor task, mathematical reasoning task, and working memory task.

For training, a 50% overlapping stride (stride = 127 samples) was used, yielding 34 windows per training file. This doubles the number of gradient steps per epoch; because the windows are highly correlated, the effective information gain is less than 2 $\times$ , but the denser sampling of the loss landscape provides a useful regularization effect on this small dataset. Validation and test windows were kept non-overlapping to avoid inflating the evaluation metric with redundant predictions from correlated windows. During evaluation, window-level softmax probabilities were averaged across all windows of a recording before taking the argmax, obtaining a single file-level prediction per recording.

#### C. Train/Validation/Test Protocol

**Initial approach** The first implementation split data at the window level. Windows from all files were pooled, shuffled, and divided into train, val and test partitions. This is incorrect, because adjacent one-second windows from the same recording are highly correlated; a window-level split places near-duplicate windows on both sides of the train/test boundary, leaking information about test recordings into training. A second leakage pattern was selecting the best epoch by *test-set* accuracy, which uses the test set as a validation proxy.

**Corrected protocol.** All splits are performed at the file level, stratified by class, before any windowing. For the intra-subject setting, the 32 training files are split into 24 for training and 8 for validation (2 per class); the 8 files in *Intra/test* are held out entirely. For the cross-subject setting, the 64 training files are split into 48 for training and 16 for validation (4 per class); each of the three test folders contains 16 files from a different unseen subject. The test folders are loaded exactly once, at the very end, for the final results. No test results are used for model or hyperparameter selection.

#### D. Model Architectures

1) *AA-CascadeNet*: First we reproduce the published architecture exactly [4]. Each one-second window mapped onto a  $20 \times 21$  scalp *mesh* (using the authors' hard-coded map), split into 10 streams of 25 time-frames. Each stem is an attention-augmented 2-D convolution. They combine convolutional feature extraction, recurrent sequence modeling, and attention, leading to a parameter count of 2,148,799. The model is designed to capture both spatial relationships among MEG sensors and temporal dependencies within neural activity recordings. To summarize it's working as a four-stage cascade: A temporal convolution first extracts local time-domain patterns and reduces dimensionality, with batch-norm, a non-linearity, and dropout after each block. A spatial convolution over the 248 sensors then captures cross-channel relationships between brain regions. The resulting feature sequence is modeled by a bidirectional LSTM, which adds long-range temporal context in both directions, and an additive attention layer aggregates it by up-weighting the most task-informative

time steps. A fully-connected softmax head maps the attention-weighted vector to the four states (rest, motor, math, working memory).

2) *Compacting AA-CascadeNet*: AA-CascadeNet achieves the high parameter count by the ten *independent* per-stream stems; each ends in a `flatten(1680) → dense(125) ≈ 210k`-parameter layer, and there are ten of them. This would mean that, for our 816 (24 files, 34 windows) training data, there are 2600 parameters per training example which is a bit extreme. We therefore compact the model, while maintaining the "cascade" structure (spatio-temporal CNN, temporal model, attention, softmax):

1. **Flat (channels × time) input instead of the 20 × 21 mesh.** We feed the raw  $248 \times 254$  array and let a single *full* spatial convolution ( $248 \times 1$ , 16 filters) learn sensor weighting directly. This removes the mesh dependency and the per-position 2-D convolution.
2. **One *shared* front-end instead of ten unshared stems.** A single EEGNet-style front-end—temporal convolution (8 filters,  $1 \times 25$ ) → batch-norm → full spatial convolution (16 filters) → ELU → average-pool → dropout—processes the whole window; weight-sharing across time is the standard, data-efficient choice and is responsible for the ( $\approx 45\times$ ) parameter reduction.
3. **A lean recurrent back-end.** One *bidirectional* LSTM (hidden 32) with additive temporal attention and a linear head, replacing two unidirectional LSTMs plus self-and-global attention.

The result is  $\approx 47k$  parameters. Smaller than the reference by a large margin, and matched to the data scale.

3) *CascadeFormer* ( $\approx 138k$  parameters): To test whether a Transformer back-end is viable on this data, we designed CascadeFormer as a single-factor variant of AA-CascadeNet: the convolutional front-end (temporal conv, full  $248 \times 16$  spatial conv, batch norm, ELU, pooling) is retained, and only the BiLSTM+attention block is replaced with a Transformer encoder ( $d_{\text{model}} = 64$ , 4 heads, 3 layers, feed-forward dimension 128, pre-LayerNorm, GELU, learned positional embeddings). A linear projection from 16 to 64 channels bridges the two stages, and classification uses mean pooling across the 63-step Transformer output followed by a linear layer ( $\approx 138k$  parameters). Relative to AA-CascadeNet this varies exactly one factor: the temporal-integration module. It kept the spatio-temporal feature extractor identical. Note that CascadeFormer is *not* a one-factor variant of the EEG Conformer; the two differ in several embedding properties at once (full vs. grouped spatial convolution, 8 vs. 40 temporal filters, lighter pooling, and a pooled vs. flattened classifier head).

4) *EEG Conformer* (*depthwise embedding, baseline Transformer*,  $\approx 87k$  parameters): We include EEG Conformer which follows Song et al. [5] as a baseline. The temporal  $1 \times 25$  convolution (40 filters) is followed by a *depthwise* spatial convolution. Leading to a grouped  $248 \times 1$  convolution ( $\text{groups} = 40$ ) in which each of the 40 temporal feature maps is spatially integrated over all 248 sensors by its own filter, with no mixing *across* feature maps. Batch normalization,

ELU, and average pooling ( $1 \times 15$ , stride 5) follow, producing a sequence of 48 tokens of dimension 40, which a Transformer encoder ( $d_{\text{model}} = 40$ , 4 heads, 4 layers, feed-forward dimension 128, pre-LayerNorm, GELU) processes; the full  $48 \times 40$  output is flattened and mapped to logits ( $\approx 87k$  parameters). This grouped spatial convolution is a deliberate deviation from the released architecture (whose spatial convolution is full,  $\text{groups} = 1$ ), adopted to keep the spatial-stage parameter count small. It is not a one-factor variant of our models. We included it as a strong, independently designed reference point

### E. Training Procedure

All models are trained as a four-class classification problem corresponding to the resting state, motor task, mathematical reasoning task, and working memory task. The categorical cross-entropy loss function was employed to measure the discrepancy between predicted class probabilities and ground-truth labels. We have early stopping (patience = 10) returning best-validation-epoch weights.

Recurrent models were optimized using the Adam optimizer, which combines adaptive learning rates with momentum-based updates. During training, mini-batches of MEG windows were presented to the network, and gradients were computed through backpropagation. Transformer models use AdamW with warmup. All models are tuned identically, cross validated by a 4-fold file-level CV on the same grid ( $\text{lr} \in \{10^{-3}, 3 \times 10^{-4}, 10^{-4}\}$ ,  $\text{dropout} \in \{0.3, 0.5\}$ )

Single-seed fragility was removed by reporting mean  $\pm$  standard deviation over 5 seeds, with same data split. We report file-level accuracy (the deployment-relevant metric) for the intra-subject test set and averaged over the three unseen cross-subject folders; chance is 0.25

The full implementation used for preprocessing, model training and evaluation is available in the project repository [7].

### F. Evaluation Protocol

Two evaluation settings were considered in this work. The first setting, referred to as *intra-subject classification*, uses recordings originating from the same subject for both training and testing. In this scenario, the model is required to learn task-specific neural activity patterns while operating within a relatively consistent subject distribution.

The second setting, referred to as *cross-subject classification*, evaluates the ability of the model to generalize across different individuals. Training is performed using recordings from a set of subjects, while testing is conducted on previously unseen subjects. This setting is substantially more challenging due to inter-subject variability in brain activity patterns and sensor measurements.

Predictions are made at the window level and aggregated to a recording-level decision: the softmax probability vectors of all windows in a recording are averaged and the argmax of the mean is taken, so that confident windows outweigh more ambiguous ones. This file-level accuracy (chance = 0.25) is reported for the held-out intra-subject test files and, in the

TABLE I  
BAND-PASS ABLATION (FILE-LEVEL ACCURACY, MEAN OVER 5 SEEDS).

| Model                   | Setting | With band-pass | No band-pass |
|-------------------------|---------|----------------|--------------|
| AA-CascadeNet (compact) | Intra   | 0.925          | 0.775        |
| AA-CascadeNet (compact) | Cross   | 0.296          | <b>0.667</b> |
| CascadeFormer (compact) | Intra   | 1.000          | 0.850        |
| CascadeFormer (compact) | Cross   | 0.271          | <b>0.696</b> |
| EEG Conformer           | Intra   | 1.000          | 0.750        |
| EEG Conformer           | Cross   | 0.392          | <b>0.667</b> |

TABLE II  
MAIN COMPARISON: FILE-LEVEL ACCURACY, MEAN  $\pm$  STD OVER 5 SEEDS, MINIMAL (NO BAND-PASS) PIPELINE. CHANCE = 0.25.

| Model                          | Intra (file)                        | Cross (file)                        |
|--------------------------------|-------------------------------------|-------------------------------------|
| AA-CascadeNet (compact)        | 0.775 $\pm$ 0.094                   | 0.667 $\pm$ 0.070                   |
| <b>CascadeFormer (compact)</b> | <b>0.850 <math>\pm</math> 0.122</b> | <b>0.696 <math>\pm</math> 0.010</b> |
| EEG Conformer (87k)            | 0.750 $\pm$ 0.137                   | 0.667 $\pm$ 0.048                   |
| AA-CascadeNet (faithful)       | 0.550 $\pm$ 0.127                   | 0.496 $\pm$ 0.036                   |
| CascadeFormer (faithful)       | 0.550 $\pm$ 0.100                   | 0.550 $\pm$ 0.047                   |

cross-subject setting, averaged over the three unseen-subject folders.

#### IV. RESULTS

##### A. Overfitting in faithful implementation

Trained to convergence (100 epochs,  $\text{lr } 10^{-4}$ ), the faithful AA-CascadeNet fits the training set *perfectly* (training accuracy  $\rightarrow 1.000$ ) while its validation accuracy does not leave chance (Fig. 1). This is a (textbook) case of overfitting by capacity. It extracts learnable features, but cannot generalize from  $\approx 800$  windows with 2.1M parameters. Multi-seed file-level accuracy stays lowest of all (Table II). These results motivate the compact designs outlined above.

##### B. Preprocessing; band-pass

(Table I) shows the differences between minimal (decimation-only) pipeline with an added  $[1, 45]$  Hz band-pass (for the right-size(d) models). Band-pass *raises* intra-subject accuracy, but *collapses* cross-subject accuracy to chance. This effect is systematic (across all models) and we therefore conclude it is a property of the *data representation*. Filtering to  $[1, 45]$  Hz preserves within-subject detail, but discards broadband power that carries the *subject-transferable* task (signal). Because the cross-subject setting is more difficult, and more meaningful, we adopt the decimation-only pipeline.

##### C. Model comparison

Table II reports the comparison on minimal pipeline. Here there are three points that stand out.

First point that stands out, is the faithful AA-CascadeNet being the worst in both setting. It overfits (Fig. 1) and recovers only partially by stopping early, showing that the capacity is the constraint. Secondly, the compact models recover the real performance. Our compact CascadeFormer is best (or tied-best) on *both* settings (.850 Intra, 0.696 cross) and most consistent in cross setting. Compact AA-CascadeNet follows behind.

The Transformer back-end edges out the recurrent back-end just marginally, but within each other’s seed variance. Thirdly, the EEG Conformer is a competitive model, but it does not dominate. Scores for both settings fall below the compact models. Compact models therefore match or slightly exceed our chosen baseline. However, the differences among them are within variance. This shows that the right-sizing and not necessarily architecture is the dominant factor for performance across all models.

##### D. Intra vs. cross-subject

Every model is better at the intra-subject task than the cross-subject task, as expected. Intra-subject data comes from one distribution, as opposed to cross-subject which requires generalizing from different individuals and distribution. The right-sized models cluster around 0.67– 0.70 file-level cross subject.

#### V. DISCUSSION

##### A. Main Findings

Our results can be separated into three separate areas:

- 1) Capacity vs. data
- 2) Bandwidth and cross-subject transferability
- 3) Front-end vs. back-end influence

##### B. Capacity vs. data

One of our clearest results is that a well-designed architecture can be unsuited for the task simply because of its size compared to the data. The reference model’s parameters memorize the training set and generalize poorly. It showed that it was tuned for a 12-subject data corpus, instead of our single-subject or two-subject data. Compacting it restores generalization, which is a useful outcome of our project.

##### C. Bandwidth and cross-subject transferability

Experimentation on the band-pass allows us to carefully point at the useful signals for cross-subject transferability. A standard MEG-pipeline intended to ‘clean’ the signal destroyed our cross-subject transfer. We think this indicates that the *discriminative* and *subject-invariant* signaling component of the brain-states therefore lives in a more broadband frequency than that the filter allows. The filter does allow for more oscillatory detail, which explains a better subject-specific decoding of the signal.

##### D. Front-end vs back-end influence

The EEG Conformer within subject accuracy is high but sensitive, and its cross-subject accuracy does not pull ahead of the compact models. We would like to point at the flattened head, whose parameters are tied to *absolute* time positions. This gives high, but sensitive, within subject capacity as it can lock onto subject-specific patterns. Our compact models used an order-invariant or attention head. These allow for more consistent accuracy.

Holding the compact front-end fixed, and swapping the LSTM-and-attention block for a Transformer in our CascadeFormer does not hurt, even marginally helps cross-subject,

Faithful AA-CascadeNet ( $\approx 2.1M$  params): train accuracy  $\rightarrow 1.0$ , validation stuck at chance

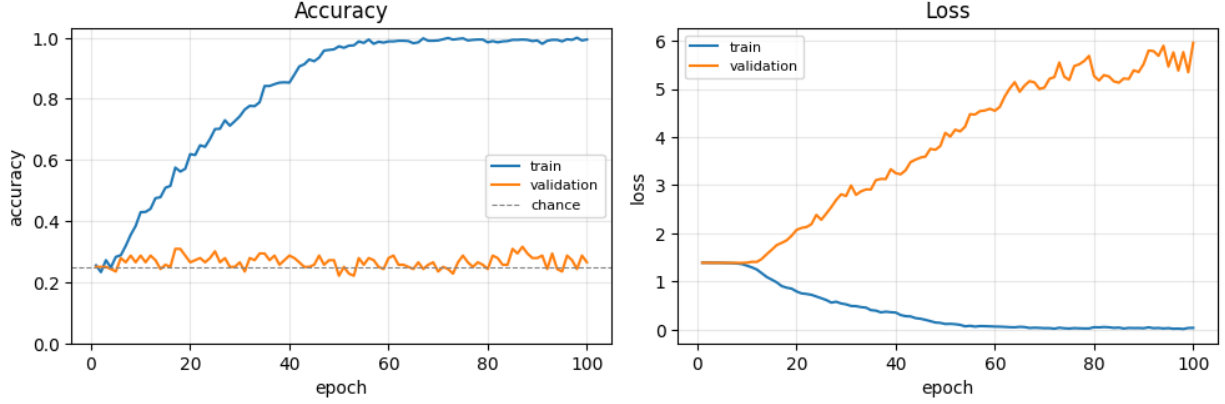


Fig. 1. Faithful AA-CascadeNet ( $\approx 2.1M$  parameters), trained to convergence (100 epochs,  $lr 10^{-4}$ ): training accuracy rises to 1.0 while validation accuracy stays at chance and validation loss diverges.

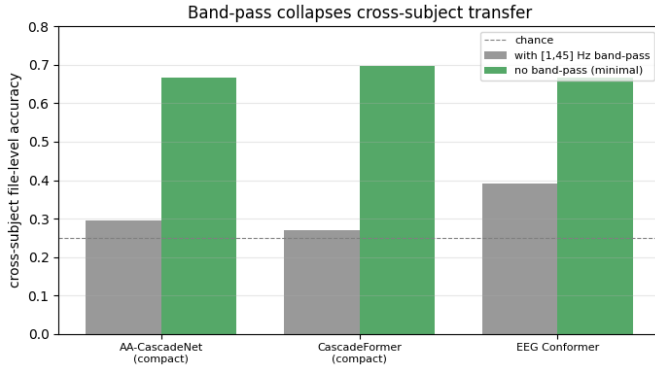


Fig. 2. Band-pass collapses cross-subject transfer: cross-subject file-level accuracy with the  $[1, 45]$  Hz band-pass versus the minimal (decimation-only) pipeline. Every model drops toward chance ( $= 0.25$ ) once band-passed.

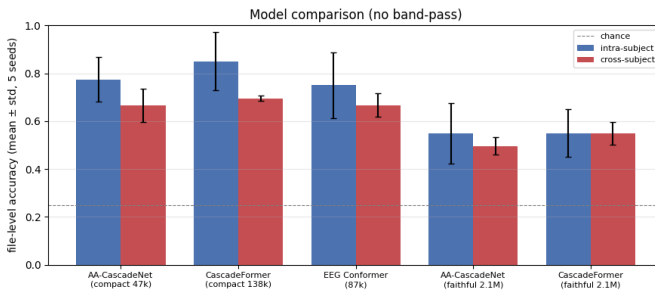


Fig. 3. Model comparison on the minimal (no band-pass) pipeline: file-level accuracy (mean  $\pm$  std over 5 seeds), intra- and cross-subject. The faithful models are worst, the compact CascadeFormer is best or tied-best on both settings. Chance = 0.25.

within variance. With only  $\approx 800$  training windows, the inductive-bias free Transformer has no advantage or penalty. The convolutional front-end does most of the work.

### E. Limitations

The main limitation of this study is the small dataset size, especially in the intra-subject setting. The intra-subject test set contains only eight recordings, so file-level accuracy has coarse resolution and should be interpreted cautiously. Cross-subject validation is also imperfect because validation files are drawn from the training subjects, which can overestimate performance on unseen subjects. Also, the AA-CascadeNet was trained under our preprocessing rather than the authors' pipeline, so findings are specific to our data handling.

### VI. CONCLUSION

In this report, we presented experiments for four-class MEG brain-state classification. Under the corrected protocol, a faithful reproduction of the AA-CascadeNet reference  $\approx 2.1M$  parameters over-fits completely on this small, subject-sparse data—training accuracy 1.0, validation at chance—so we compacted it (sharing the front-end across time and replacing the mesh/multi-stream design with a flat CNN front-end). The resulting compact CNN-BiLSTM model ( $\approx 47k$  parameters) and one-factor CNN-Transformer CascadeFormer ( $\approx 138k$  parameters) recover genuine generalization, reaching 0.67–0.70 file-level accuracy on unseen subjects; the compact CascadeFormer is the best and most consistent model overall and nominally matches or exceeds a strong published Transformer baseline (EEG Conformer) on both settings. We also found that a routine  $[1, 45]$  Hz band-pass, far from helping, pushes cross-subject accuracy toward chance, so we adopt a minimal broadband pipeline. The unifying message is that matching capacity (and bandwidth) to the data is what governs cross-subject MEG decoding at this data scale. Future work should pursue subject-invariant representations, leave-subjects-out evaluation over more subjects, and data augmentation.

### REFERENCES

- [1] Lawhern, V. J., Solon, A. J., Waytowich, N. R., Gordon, S. M., Hung, C. P., and Lance, B. J. (2018). EEGNet: a compact convolutional neural

- network for EEG-based brain-computer interfaces. *Journal of neural engineering*, 15(5), 056013. <https://doi.org/10.1088/1741-2552/aace8c>
- [2] Schirrmester, R. T., Springenberg, J. T., Fiederer, L. D. J., Glasstetter, M., Eggenberger, K., Tangermann, M., Hutter, F., Burgard, W., and Ball, T. (2017). Deep learning with convolutional neural networks for EEG decoding and visualization. *Human brain mapping*, 38(11), 5391–5420. <https://doi.org/10.1002/hbm.23730>
  - [3] Zubarev, I., Zetter, R., Halme, H.-L., and Parkkonen, L. (2019). Adaptive neural network classifier for decoding MEG signals. *NeuroImage*, 197, 425–434. [10.1016/j.neuroimage.2019.04.068](https://doi.org/10.1016/j.neuroimage.2019.04.068)
  - [4] Alaoui Abdellaoui, A., García Fernández, A., Şahinli, S., and Mehrkanoon, S. (2020). Deep brain state classification of MEG data. *arXiv preprint arXiv:2007.00897*. <https://doi.org/10.48550/arXiv.2007.00897>
  - [5] Song, Y., Zheng, Q., Liu, B., and Gao, X. (2023). EEG Conformer: Convolutional Transformer for EEG decoding and visualization. *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, 31, 710–719. <https://doi.org/10.1109/tnsre.2022.3230250>
  - [6] Roy, Y., Banville, H., Albuquerque, I., Gramfort, A., Falk, T. H., and Faubert, J. (2019). Deep learning-based electroencephalography analysis: a systematic review. *Journal of neural engineering*, 16(5), 051001. [10.1088/1741-2552/ab260c](https://doi.org/10.1088/1741-2552/ab260c)
  - [7] C. Inal, A. Abbasli, Y. Bergstra, and T. van der Meijden, *DeepLearningAssignment2*, GitHub repository, 2026. [Online]. Available: <https://github.com/Draenaii/DeepLearningAssignment2>. Accessed: Jun. 17, 2026.